Отчёт об исправлении 15-кратного расхождения MBS-raw vs MBS-old

MBS-fast project

24 марта 2026

1 Описание проблемы

При сравнении нового оптимизированного кода (MBS-raw/MBS-fast) со старым бинарником (MBS-old) в режиме фиксированной ориентации `-fixed 45 30` обнаружено 15-кратное расхождение в значениях M_{11}:

Код	$M_{11}(0°)$	Параметры
MBS-old	5.76×10^5	`-fixed 45 30 -grid 0 25 360 25`
MBS-raw (новый)	8.93×10^6	те же + `-legacy_sign`

Отношение $\approx 15.5\times$. Флаг `-legacy_sign` (знак Френеля) влияния не оказал.

2 Методика диагностики

2.1 Шаг 1: Инкогерентный режим

Запуск обоих бинарников с флагом `-incoh` (мюллеровские матрицы суммируются поштучно по лучам, без когерентного сложения амплитуд):

Тест	Макс. отн. разница	Вывод
`-fixed 0 0 -incoh`	$< 10^{-6}$	Поштучная дифракция идентична
`-fixed 0 0` (когерент.)	$\sim 5\%$	Фазы отличаются

Вывод: амплитуды $|J_{\text{beam}}|$ идентичны, расхождение только в фазах при когерентном суммировании.

2.2 Шаг 2: Обнаружение отсутствия поворота частицы

Сравнение кода `TracerPO::TraceFixed` в MBS-old и MBS-raw:

```
// MBS-old:                                                    !
void TracerPO::TraceFixed(double beta, double gamma) {
    double b = DegToRad(beta);
    double g = DegToRad(gamma);
    m_scattering->ScatterLight(b, g, outBeams);  //

    m_handler->HandleBeams(outBeams, 1);
}

// MBS-raw (           ):
void TracerPO::TraceFixed(double beta, double gamma) {
```

```
    double b = DegToRad(beta);
    double g = DegToRad(gamma);
    m_particle->Rotate(b, g, 0);                    // <--

    m_scattering->ScatterLight(b, g, outBeams);
    m_handler->HandleBeams(outBeams, 1);
}
```

В `ScatteringConvex::ScatterLight` вызов `m_particle->Rotate()` **закомментирован**. Старый код всегда считает рассеяние для ориентации по умолчанию, игнорируя аргументы `-fixed`.

Это баг старого кода. Для ориентационного усреднения (`TracerPOTotal`) баг не проявляется, т.к. `Rotate` вызывается перед `ScatterLight(0, 0, ...)`.

2.3 Шаг 3: Остаточное расхождение 5% при –fixed 0 0

После устранения фактора поворота (тест при $\beta = 0, \gamma = 0$) осталось $\sim 5\%$ расхождение в когерентном режиме. Инкогерентный — точное совпадение.

Причина: вызов `Rotate(0, 0, 0)` в новом коде запускает `SetDParams()`, которая инициализирует `d_param` (4-й компонент `Point3f::coordinates[3]`).

3 Найденные баги

3.1 Баг 1: Неинициализированный d_param

`Point3f` имеет пустой конструктор по умолчанию:

```
struct Point3f {
    float coordinates[4];
    Point3f() {}                    //                        coordinates
        [3]!
    Point3f(float x, float y, float z) {
        coordinates[0] = x;
        coordinates[1] = y;
        coordinates[2] = z;         // coordinates[3]                !
    }
};
#define d_param coordinates[3]
```

Во всех конструкторах частиц:

```
SetDefaultNormals();    //                                    (cx,cy,cz),
              d_param
Reset();                //                   defaultFacets -> facets
SetDefaultCenters();    //
// SetDParams()                            -> d_param                 !
```

Только `Rotate()` вызывает `SetDParams()`.

Исправление

Добавлен `SetDParams()` после `Reset()` во все 8 конструкторов частиц:

- `Hexagonal.cpp`
- `ConcaveHexagonal.cpp`
- `Bullet.cpp`

- `BulletRosette.cpp`
- `Droxtal.cpp`
- `HexagonalAggregate.cpp`
- `CertainAggregate.cpp`
- `Particle.cpp` (загрузка из файла)

3.2 Баг 2: Смешение fast_exp_im и exp_im

В fallback-функциях дифракции (`DiffractIncline`, `DiffractInclineFast`) в одном выражении смешивались `fast_exp_im` (полиномиальный sincos) и `exp_im` (glibc sincos):

```
//          :
tmp = (fast_exp_im(kCi*p2.x) - exp_im(kCi*p1.x)) / Ci;
s += fast_exp_im(m_waveIndex*B*bi) * tmp;

//            :
tmp = (exp_im(kCi*p2.x) - exp_im(kCi*p1.x)) / Ci;
s += exp_im(m_waveIndex*B*bi) * tmp;
```

Аналогично исправлены `ComputeFnJones`, `ApplyDiffraction`, `ApplyDiffractionFast`, `ApplyDiffractio

Примечание: инлайн-путь HandleBeams по-прежнему использует `fast_sincos` повсеместно (нет смешения), что корректно и быстро.

4 Результаты валидации

Тест	Режим	Макс. отн. разн.	Примечание
`-fixed 0 0`	incoh	3.3×10^{-5}	d_param фикс
`-fixed 0 0`	coh, без Rotate	$< 10^{-6}$	точн. совп.
`-fixed 0 0`	coh, с Rotate	$\sim 5\%$	d_param: мусор vs правильный
`-fixed 45 30`	old binary	всегда = ориентация 0 (баг)	
`-fixed 45 30`	new binary	корректный поворот	

5 Влияние на рабочие режимы

Режим	Затронут?	Причина
`-random` (TracerPOTotal)	Нет	Rotate() всегда вызывается
`-sobol`	Нет	То же
`-adaptive / -auto`	Нет	То же
`-fixed` (TracerPO)	**Да**	Добавлен Rotate + d_param
`-montecarlo` (TracerPOTotal)	Нет	Rotate() всегда вызывается

Для всех рабочих режимов ориентационного усреднения (random, sobol, adaptive, auto) исправление не меняет результатов, т.к. `TracerPOTotal` всегда вызывает `Rotate()` перед `ScatterLight()`, что инициализирует `d_param`.

6 Заключение

1. Исходное 15-кратное расхождение — не баг HandleBeams, а **сравнение разных ориентаций**: старый код не поворачивает частицу в режиме `-fixed`.

2. Остаточное 5% расхождение в когерентном режиме — от неинициализированного `d_param`, исправлено во всех конструкторах.

3. Инлайн-оптимизации HandleBeams (batched sincos, precomputed edges, theta-coefficients) **верифицированы**: дают идентичный результат поштучно.

4. Fallback-функции приведены к единообразному использованию `exp_im`.